

Laboratorio di Linguaggi

M. Pasqua

A.A. 2025/2026

Progetto

Il progetto di laboratorio consiste nello sviluppare un linguaggio di programmazione che soddisfi dei requisiti strutturali e funzionali minimi. Nello specifico è necessario:

1. Definire la grammatica context-free del linguaggio ed implementarla in ANTLR;
2. Definire la semantica del linguaggio ed implementarla tramite un interprete basato sul pattern visitor e ANTLR¹;
3. Fornire alcuni programmi d'esempio che mostrino i costrutti e le caratteristiche principali del linguaggio;
4. Produrre la documentazione del linguaggio, che ne descrive l'utilizzo.

La valutazione del progetto avverrà a seguito della discussione orale del linguaggio sviluppato, e consisterà in un punteggio:

- Da 0 a 4, se il progetto viene consegnato al primo appello d'esame (17/06/2026);
- Da 0 a 3, se il progetto viene consegnato ad un appello d'esame successivo al primo.

Il progetto può essere sviluppato da soli o in gruppo (consigliato), di al più **tre** persone, e può essere consegnato e discusso una sola volta per anno accademico.

Requisiti del Linguaggio

Definire un linguaggio imperativo tipizzato (staticamente o dinamicamente) con variabili e assegnamenti. Il linguaggio può essere generico (general-purpose), oppure specifico per un determinato dominio applicativo (DSL).

In ogni caso, il linguaggio deve supportare almeno le espressioni aritmetico-logiche di base (addizione, sottrazione, moltiplicazione, divisione, uguaglianza, congiunzione, disgiunzione, negazione) con precedenza, almeno un costrutto iterativo (ciclo), almeno un costrutto condizionale (scelta condizionata), blocchi di codice definiti da due simboli delimitatori e visibilità riservata (con o senza shadowing), almeno tre tipi differenti oltre ai booleani e le stringhe, ed un comando per la stampa a video. *Il linguaggio deve gestire in modo programmatico eventuali errori a tempo d'esecuzione.*

Inoltre, il linguaggio deve supportare almeno **n** delle seguenti funzionalità avanzate, dove **n** è il numero di partecipanti al progetto. Ogni funzionalità avanzata è assegnata ad un livello di complessità crescente: 😊 << 😊😊 << 😊😊😊.

¹L'implementazione può essere scritta in uno qualsiasi dei linguaggi supportati da ANTLR (linguaggi supportati: <https://www.antlr.org/download.html>), tranne JavaScript.

Strutture Dati ☹ Supporto per array di valori a dimensione fissa o variabile. Gli elementi di un array sono acceduti tramite indice numerico, ad esempio `array[2]` accede al terzo elemento dell'array `array`. *In alternativa*, supporto per record di valori a struttura fissa o variabile. Gli attributi di un record sono acceduti tramite notazione puntata, ad esempio `record.campo` accede all'attributo `campo` del record `record`.

Funzioni ☹ Supporto per la definizione e chiamata di funzione con parametri e valore di ritorno. Il passaggio dei parametri può avvenire per valore o riferimento. Deve essere possibile chiamare funzioni ricorsivamente. Le funzioni vengono dichiarate in una sezione apposita del programma, non eseguibile.

Conversione di Tipo ☹ Supporto per la conversione implicita ed esplicita dei tipi. La gerarchia dei tipi deve contenere almeno una catena di sottotipaggio (ad esempio, $\mathbb{N} \subsetneq \mathbb{Z} \subsetneq \mathbb{Q}$) che non coinvolga tipi numerici. La gerarchia dei tipi deve contenere un tipo radice `Any` dal quale derivano tutti i tipi del linguaggio.

Flusso di Controllo Incondizionato ☹ Supporto per la versione short-circuit delle espressioni logiche, ad esempio durante la valutazione dell'espressione `false && x != 3` non si deve valutare la sotto-espressione `x != 3`, in quanto non necessario. Il linguaggio deve fornire anche la versione non short-circuit delle espressioni logiche. Supporto per due comandi che permettono l'uscita prematura da un blocco di codice. Il primo, restituisce l'esecuzione al blocco di codice appena più esterno, mentre il secondo termina l'esecuzione del programma.

Non Determinismo ☹ Supporto per un costrutto di scelta non deterministica, che esegue un comando tratto da una lista con una probabilità proporzionale alla dimensione della lista. Ad esempio, la valutazione del comando `x = 1 @ x = 2` esegue il comando `x = 1` con probabilità $1/2$ (o il comando `x = 2` con probabilità $1/2$), mentre la valutazione del comando `x = 1 @ x = 2 @ x = 3` esegue il comando `x = 1` con probabilità $1/3$. Supporto per un comando che cattura l'input da tastiera.

Valutazione Pigra ☹ Supporto per la valutazione pigra (lazy) delle espressioni tramite specifico modificatore per gli assegnamenti. In un assegnamento pigro, l'espressione non viene valutata durante la valutazione dell'assegnamento, ma solo quando la variabile assegnata viene effettivamente usata (se questo avviene). Ad esempio, nella valutazione del comando `y = 4 ; lazy x = y + 3 ; y = 0 ; print x` la variabile `x` viene assegnata con il valore 7 solamente quando viene valutato il comando `print x`, non prima.

Inlining di Codice ☹ Supporto per un comando che esegue del codice scritto in uno dei linguaggi visti a lezione. Gestire lo scambio di variabili tra il linguaggio chiamante ed il linguaggio chiamato. Il linguaggio chiamato deve eseguire il corrispondente interprete (già implementato) sul codice fornito. Ad esempio, la valutazione del comando `Integer x = 3 ; inline { print toString(x) ; x = 5.2 }` richiama l'interprete del linguaggio `Imp` sul programma `'print toString(x) ; x = 5'` e stampa a video il numero 3. La variabile `x` nel linguaggio chiamante non viene modificata.

Flusso di Controllo Condizionato ☹ Supporto per un costrutto iterativo con possibilità di uscita prematura dall'iterazione tramite un apposito comando. Il costrutto prevede un blocco di codice da eseguire se e solo se la condizione d'iterazione diventa falsa (quindi il comando di uscita prematura non viene eseguito). Ad esempio, nel seguente comando `for i from 2 to (n-1) { if (n % i == 0) break } { print`

`"Not prime!"}` la stampa a video avviene solo quando `n` non è un numero primo. Viceversa, se `n` è un numero primo nulla viene stampato ed il ciclo viene interrotto, ad un certo punto, dal comando `break`. *In alternativa*, supporto per un costrutto condizionale a scelta multipla (switch), nel quale l'espressione di controllo può assumere uno dei valori tra quelli a scelta. Solo il blocco di codice relativo ad una delle scelte viene eseguito. Il costrutto può avere o meno una scelta predefinita (default) da eseguire quando nessuna scelta è percorribile.

Zecchero Sintattico ☺ Supporto per operatori di pre e post incremento o decremento unitario, adottando la semantica degli operatori analoghi di Java. Supporto per comandi di assegnamento composto per le operazioni aritmetiche di base. Gli assegnamenti composti preservano la semantica degli assegnamenti semplici. Ad esempio, l'assegnamento composto `x += 5` equivale all'assegnamento semplice `x = x + 5`. Supporto per un operatore ternario per le espressioni analogo all'operatore `cond ? exp1 : exp2` di Java, il quale restituisce la valutazione di `exp1` quando la valutazione di `cond` risulta vera. Restituisce la valutazione di `exp2` quando la valutazione di `cond` risulta falsa. Supporto di espressioni all'interno delle stringhe del linguaggio. Ad esempio, il comando `print "Risultato: ${x + y}"` stampa a video 'Risultato: 5' se le variabili `x` e `y` contengono i valori 3 e 2, rispettivamente.

Classi e Oggetti ☺ Supporto per la definizione di semplici classi e la creazione di oggetti come istanze di tali classi. Nelle classi è possibile dichiarare campi e metodi, specificando due livelli di visibilità per entrambi: visibilità limitata internamente alla classe (privati) e visibilità interna ed esterna alla classe (pubblici). I metodi di una classe possono essere chiamati, se visibili, dal codice che detiene una referenza ad un oggetto istanza di tale classe, tramite notazione puntata. Ad esempio, `Dot dot = new Dot() ; print dot.getX()` istanzia un oggetto `dot` della classe `Dot` e poi chiama il metodo `getX` della classe sull'oggetto `dot`. I metodi di una classe sono essenzialmente funzioni con parametri, passati per valore, ed eventualmente un valore di ritorno. I parametri dei metodi possono anche essere referenze ad oggetti. All'interno dei metodi è possibile dichiarare variabili locali al metodo, visibili solo al metodo stesso, oppure istanziare oggetti. Le classi non hanno costruttori espliciti. Il linguaggio non prevede ereditarietà.

Documentazione

La documentazione consiste in un file `doc.md` scritto in Markdown² che descrive il linguaggio. Nello specifico, la documentazione introduce le caratteristiche principali del linguaggio, le scelte progettuali adottate, ed il contesto applicativo per il quale è stato sviluppato. Una possibile struttura per la documentazione è la seguente.

Introduzione Presentazione del linguaggio, descrizione delle caratteristiche principali, e discussione del contesto d'applicazione.

Guida Rapida Descrizione dei requisiti d'installazione ed esecuzione, spiegazione dei comandi per generare ed eseguire l'interprete del linguaggio, e presentazione di uno o programmi d'esempio (hello world).

Sintassi Descrizione dei costrutti e dei tipi di dato principali del linguaggio, discussione di eventuali limitazioni sintattiche, e riassunto delle regole lessicali principali (si può fare riferimento al file `.g4` della grammatica).

²Documentazione Markdown: <https://www.markdownguide.org>.

Semantica Descrizione delle principali scelte progettuali (ad esempio, tipologia di controllo dei tipi, visibilità delle variabili, tipologia di valutazione delle espressioni, tipologia di passaggio dei parametri, gestione degli errori di tipo e d'esecuzione), formalizzazione delle regole di transizione della semantica operativa del linguaggio per i costrutti più significativi, e presentazione della gerarchia dei tipi e della strategia di conversione.

Implementazione Breve spiegazione di come è stato implementato il visitor per il linguaggio, le eventuali difficoltà tecniche incontrate e come sono state risolte.

Programmi di Test Presentazione di alcuni programmi complessi che mostrino le funzionalità principali del linguaggio, con output atteso.

Consiglio per la scrittura All'interno di un file Markdown è possibile inserire codice $\text{\LaTeX}$ tramite una coppia di dollari \$ (codice in linea) o una coppia di doppi dollari \$\$ (codice a larghezza pagina). Ad esempio, il seguente codice

```
$$
\text{\text{Seq}}\sim\frac{
  (\sigma, com_1) \rightarrow (\sigma', com'_1)
}{
  (\sigma, com_1 ; com_2) \rightarrow (\sigma', com'_1 ; com_2)
}
$$
```

visualizza nella pagina quanto segue:

$$\text{Seq} \frac{(\sigma, com_1) \rightarrow (\sigma', com'_1)}{(\sigma, com_1 ; com_2) \rightarrow (\sigma', com'_1 ; com_2)}$$

Consegna e Valutazione

La consegna del progetto avviene tramite un file `.zip` contenente il codice e la documentazione del linguaggio, inviato via email al docente (michele.pasqua@univr.it) entro la data di un appello d'esame. Nello specifico, deve contenere:

- Un file `.g4` che descrive la grammatica del linguaggio;
- Una cartella `source` contenente i file sorgente dell'interprete del linguaggio;
- Un file `doc.md` che rappresenta la documentazione del linguaggio;
- Una cartella `programs` contenente i programmi d'esempio scritti nel linguaggio.

Dalla documentazione deve essere possibile capire come generare l'interprete a partire dai file sorgente, e come eseguirlo sui programmi d'esempio.

La valutazione del progetto, a seguito della discussione orale, si baserà sui seguenti criteri. La proposta di **lode** (da considerare qualora il voto complessivo del corso, comprendente anche la parte di teoria, sia 30) viene valutata solo se il linguaggio presentato implementa almeno una funzionalità avanzata di complessità almeno ☹.

Correttezza dell'implementazione, e coerenza con la documentazione fornita.

Complessità del linguaggio, in relazione alle funzionalità avanzate scelte.

Chiarezza espositiva della documentazione e della discussione orale.

Originalità del linguaggio, dal punto di vista architetturale e/o di dominio applicativo.